BUILD RECORD · WRITTEN AFTER THE FACT, FROM WHAT ACTUALLY HAPPENED

Local setup & deployment

How this project runs on a laptop, how it got onto the internet, and the eleven things that went wrong on the way — so none of them has to be rediscovered.

Repository	github.com/KSShreyash/AI_Workspace_Assistant — branch main
Supabase ref	pcrujlovqecgzcuzwqtw · ap-south-1 · free
Supabase URL	https://pcrujlovqecgzcuzwqtw.supabase.co
OAuth callback	https://pcrujlovqecgzcuzwqtw.supabase.co/auth/v1/callback
API	https://ai-workspace-assistant-api.onrender.com
Frontend	https://ai-workspace-assistant.pages.dev
Migrations	15, through 20261009114228_revoke_data_api_access.sql
Tests	400 backend · 197 frontend

NO SECRETS IN THIS DOCUMENT

Database password, Groq key, Azure secret and Google secret are deliberately absent. They live in `backend/.env` (local) and `backend/.env.cloud` (cloud), both gitignored, and in the Render dashboard. This file is safe to share; those two are not.

PART ONE

Running it locally

Everything local, nothing touching the cloud project. This is what the test suite runs against.

THE RULE THAT GOVERNS THE WHOLE SETUP

`profiles.id` has a foreign key to `auth.users(id)`, so the application database and the Auth service **must be the same Supabase instance**. A local database paired with cloud Auth cannot work — a user id minted by cloud Auth violates the foreign key against the local `auth.users`. JWT verification follows `SUPABASE_URL` for the same reason.

So there are two self-consistent worlds, never mixed. Switch `DATABASE_URL`, `SUPABASE_URL` and the keys together, or not at all.

Prerequisites

Docker Desktop, Node 22, Python 3.11, git. Docker must be running first.

1 Install dependencies

```
npm install                          # root: the Supabase CLI
cd backend
.venv/Scripts/python.exe -m pip install -r requirements.txt
cd ../frontend && npm install
```

TRAP 1 — NODE_MODULES CAN BE INCOMPLETE

`next dev` returned HTTP 500 on every page with *"Cannot find module 'framer-motion'"*. The package was in `package.json` but not installed. `npm install` fixed it. Worth running before assuming the code is broken.

2 Start Postgres

```
# database only — enough for the whole test suite
npx supabase start --debug -x gotrue,realtime,storage-api,imgproxy,kong,\
mailpit,postgrest,postgres-meta,studio,edge-runtime,logflare,vector,supavisor

# full stack — needed for browser sign-in
npx supabase start --debug -x realtime,storage-api,imgproxy,mailpit,\
edge-runtime,logflare,vector,supavisor
```

TRAPS 2 AND 3 — BOTH SILENT

Always pass `--debug` on the first run. Without a TTY the CLI's progress renderer buffers everything, so `supabase start` prints nothing and pulls nothing for many minutes while appearing to work.

One invalid name voids the entire `-x` flag. The CLI does not error on an unknown service; it silently ignores the whole list and pulls all thirteen images. It is `postgres-meta`, not `pg-meta`.

3 Apply migrations

```
npx supabase db reset     # replays all 15 from empty
npx supabase status       # prints the local keys
```

Re-run `db reset` after any migration change. It is the only check that a fresh deployment will work — applying a migration to an already-migrated database proves far less.

4 Configure the backend

Copy `backend/.env.example` to `backend/.env`. The local database URL is fixed:

```
DATABASE_URL=postgresql://postgres:postgres@127.0.0.1:54322/postgres
SUPABASE_URL=http://localhost:54321
```

Generate the two secrets you own, once:

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

> TRAP 4 — FERNET_KEY CAN NEVER BE ROTATED
>
> Every stored access and refresh token becomes permanently undecryptable and every mailbox needs reconnecting by hand. There is no recovery path. The same key is deliberately used for local and cloud so a mailbox connected in either stays readable in both.

5 Run the tests — the real checkpoint

```
cd backend && .venv/Scripts/python.exe -m pytest -q   # 400 passed
cd ../frontend && npx vitest run                      # 197 passed
npx tsc --noEmit && npx eslint .                      # silent
```

The backend suite needs no network at all. If tests *error* rather than fail, `DATABASE_URL` is not pointing at the local stack or the container is not running.

6 Run both services

```
# terminal 1
cd backend && .venv/Scripts/uvicorn app.main:app --reload --port 8000

# terminal 2
cd frontend && npm run dev            # http://localhost:3000
```

`localhost:8000/health` returns `{"status":"ok"}`. Sign-in additionally needs the OAuth apps from Part two and the `[auth.external.*]` blocks enabled in `supabase/config.toml`.

> TRAP 5 — THE FRONTEND MUST BE SERVED ON PORT 3000
>
> Serving the built export on `:3100` made every API call fail CORS silently. The backend allows `http://localhost:3000` literally, and origins are compared as strings. Same reason there must be **no space after the comma** in `CORS_ALLOW_ORIGINS`.

7 Test the production build, not just dev

```
cd frontend && npm run build && npx serve out -l 3000
```

`next.config.ts` sets `output: "export"`, so what ships is static files with no server. Some bugs — hydration mismatches in particular — appear only there.

Stopping it, so nothing keeps running

Three things hold resources: the two app servers, the Postgres container, and Docker Desktop itself. Stop them in that order — the Supabase CLI needs the Docker engine alive to stop its own container.

1 Stop the app servers

Ctrl-C in each terminal. If they were backgrounded and the terminal is gone, kill by port:

```
# PowerShell
Get-NetTCPConnection -LocalPort 3000,8000 -State Listen -ErrorAction SilentlyContinue |
  Select-Object -Expand OwningProcess -Unique |
  ForEach-Object { Stop-Process -Id $_ -Force }
```

2 Stop Postgres

```
npx supabase stop     # the data volume is preserved
```

> ORDER MATTERS
>
> Run this *before* quitting Docker Desktop. With the engine already down the command fails with `StopListError … open //./pipe/ dockerDesktopLinuxEngine`, because the CLI cannot list the container it is trying to stop. Harmless — the container died with the engine — but it looks like a failure.

3 Quit Docker Desktop

From the system tray. This is what actually frees the memory: the Linux VM keeps running after the containers stop.

4 Verify, rather than assume

```
# every port should be free
Get-NetTCPConnection -LocalPort 3000,3100,8000,54321,54322 -State Listen `
  -ErrorAction SilentlyContinue

# engine down, and nothing of ours left
docker info
Get-Process node,python3.11 -ErrorAction SilentlyContinue
```

All three should come back empty. A stray `next dev` surviving a closed terminal is the usual culprit.

> NOTHING IS LOST BY STOPPING
>
> `supabase stop` leaves the named volume (`supabase_db_AI_Workspace_Assistant`) in place, so `supabase start` restores the database exactly as it was — migrations and any seeded data included. Only `supabase stop --no-backup`, or deleting the volume, discards it.
>
> None of this touches the cloud: the Supabase project, the Render service and Cloudflare Pages keep running independently.

PART TWO

Deploying it

Five providers, in dependency order. Each produces a value the next one needs.

1 Supabase project

Created through the Supabase MCP, then driven by the CLI:

```
npx supabase login
npx supabase link --project-ref pcrujlovqecgzcuzwqtw
npx supabase db push
npx supabase db advisors --linked     # expect: no issues found
```

TRAP 6 — MIXED TOOLING SPLITS THE MIGRATION HISTORY

Three migrations were applied over MCP before it disconnected. It recorded them under its own generated version numbers, so `db push` would have re-applied them and failed on "table already exists". Reconciled with:

```
npx supabase migration repair --linked --status reverted <mcp versions>
npx supabase migration repair --linked --status applied  <file versions>
```

Pick one tool for migrations and stay with it. The CLI is the reliable one.

2 Supabase Auth settings — three, all in the dashboard

SETTING	WHERE	VALUE
Allow manual linking	Auth → Providers	`on`
Site URL	Auth → URL Config	`https://ai-workspace-assistant.pages.dev`

Redirect URLs	Auth → URL Config	`…pages.dev/callback`
		`http://localhost:3000/callback`
		`http://127.0.0.1:3000/callback`

TRAPS 7 AND 8 — BOTH FOUND BY MEASURING, NOT GUESSING

Manual linking is off by default. Without it `linkIdentity()` is rejected, so "Add a mailbox" fails — the entire multi-account feature.

Redirect URLs start empty. Signing in with Google then redirected to `http://localhost:3000/?code=…` — the request to return to the Pages domain was not allow-listed, so Supabase fell back to Site URL, which was still localhost. OAuth itself had worked; only the last hop was wrong.

`npx supabase config diff` reports all three against the cloud project. Do not run `config push` to fix them: it would also push local-development values — email confirmations off, OTP length 6, MFA disabled — which is a real security downgrade.

3 Microsoft Entra app registration

- Supported account types: **multitenant + personal Microsoft accounts**, or outlook.com and hotmail cannot sign in.
- Web redirect URIs: the Supabase callback *and* `http://localhost:54321/auth/v1/callback` — one app serves both environments.
- Delegated Graph permissions: `User.Read`, `Mail.Read`, `Calendars.ReadWrite`, `Chat.Read`, `OnlineMeetings.ReadWrite`, plus `offline_access`.
- Enable Azure in Supabase → Providers with the client ID and secret.

> TRAP 9 — COPY THE VALUE, NOT THE SECRET ID
>
> On the client-secret screen the **Value** column is the secret and is shown once. The Secret ID looks like a GUID and is the wrong thing. The secret also expires — when it does, every mailbox fails to refresh and shows as "needs reconnecting", which reads like revoked consent rather than an expired secret.

4 Google Cloud OAuth client

- Enable **both** the Gmail API and the Google Calendar API. Missing either gives a 403 the app records as `not_available` and then stops retrying — it fails quietly.
- Google Auth Platform → Branding, then Audience → **External**. The old "OAuth consent screen" page now redirects here.
- Audience → **Test users** → add your own Google address.
- Credentials → OAuth client ID → Web application, with the Supabase callback.
- Skip the Data Access / scopes tab — the app requests its scopes at sign-in.

> TRAP 10 — TESTING STATUS MEANS 7-DAY REFRESH TOKENS
>
> Google's own docs: a project with an external consent screen in *Testing* is "issued a refresh token expiring in 7 days". So every connected Gmail mailbox stops syncing weekly until the app is published, and only listed test users can authorise it at all.
>
> Publishing removes both limits but `gmail.readonly` is a **restricted** scope, which cannot be used in production without Google's security review. Staying in Testing is the right call for a personal project.

5 Render — the backend

New → Blueprint from the repo. `render.yaml` defines the service as `ai-workspace-assistant-api`; every variable marked `sync: false` is copied from `backend/.env.cloud`. `CORS_ALLOW_ORIGINS` has a literal value in the blueprint, so Render never prompts for it.

> **TRAP 11 — THE DIRECT DATABASE HOST IS IPV6-ONLY**
>
> `db.<ref>.supabase.co` publishes only an AAAA record. Render's free tier is IPv4, so a direct connection fails there as *"Network is unreachable"*, which reads like the database is down rather than unreachable by that route. Use the **session pooler**:
>
> ```
> postgresql://postgres.<ref>:<password>@aws-0-ap-south-1.pooler.supabase.com:5432/postgres
> ```
>
> Session mode (5432), **not** transaction mode (6543): asyncpg uses prepared statements, which pgbouncer's transaction mode does not support.
>
> And percent-encode the password. Ours contained `@`, which is the delimiter between credentials and host — pasted raw it parses the host wrong. `@` becomes `%40`.

Verify more than the health check:

```
curl .../health                       # {"status":"ok"}
curl .../api/emails                   # 401 — auth is enforced
curl -X POST .../api/sync/run \
  -H "X-Sync-Secret: <secret>"        # counts — proves it reaches Postgres
```

6 Cloudflare Pages — the frontend

SETTING	VALUE
Framework preset	`Next.js (Static HTML Export)`
Root directory	`frontend`
Build command	`npm run build`
Output directory	`out`

Three variables, from `frontend/.env.cloud`:

```
NEXT_PUBLIC_SUPABASE_URL=https://pcrujlovqecgzcuzwqtw.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_…
NEXT_PUBLIC_API_BASE_URL=https://ai-workspace-assistant-api.onrender.com
```

> **THESE ARE INLINED AT BUILD TIME**
>
> Not read at runtime. Changing one in the dashboard does nothing until a **rebuild** — a redeploy without one keeps the old values compiled into the bundle. Set all three before the first build.
>
> The publishable key is safe in public; it is designed for browser code and the database grants it nothing. The service-role key is never needed — it is not read anywhere in the app.

No scheduled sync, deliberately

> WHY THE GITHUB ACTIONS CRON WAS DELETED
>
> It called the API every 15 minutes, which keeps a Render free instance awake continuously — and the free plan allows roughly **750 instance-hours a month across the whole account**. One always-on service consumes essentially all of it, leaving nothing for anything else.
>
> Sync now runs on sign-in and from the **Sync now** button, both of which wake the instance; it sleeps again about 15 minutes later. `POST /api/sync/run` still exists for any external scheduler. If you re-enable one, prefer a few times a day: four calls costs about an hour of uptime daily instead of twenty-four.

PART THREE

Reference

Where each value lives

FILE	TRACKED?	HOLDS
`backend/.env`	no	Local stack. Points at 127.0.0.1:54322.
`backend/.env.cloud`	no	Cloud values, for pasting into Render.
`backend/.env.example`	yes	Local template.
`backend/.env.cloud.example`	yes	Cloud template, with the pooler explained.
`frontend/.env.local`	no	The three NEXT_PUBLIC values, local.
`frontend/.env.cloud`	no	The same three, cloud.

> **THE GITIGNORE NEEDED WIDENING**
>
> It named `backend/.env` exactly, so `backend/.env.cloud` would have been committed with live credentials. It is now `backend/.env*` with an exception for `*.example`. Worth checking with `git check-ignore -v <file>` before writing any secret to a new filename.

Symptoms and causes

SYMPTOM	CAUSE
`supabase start` silent for minutes	Progress renderer buffering. Add `--debug`.
All 13 images download despite `-x`	One invalid service name voids the whole flag.
Tests *error* rather than fail	No database. Check the container and `DATABASE_URL`.
Every page 500s in `next dev`	Incomplete `node_modules`. Run `npm install`.
CORS failures with nothing logged	Wrong port, or a space after the comma in the origins list.
Render: "Network is unreachable"	Direct DB host is IPv6-only. Use the session pooler.
Prepared-statement errors on Render only	Transaction pooler (6543). asyncpg needs session (5432).
Service exits with `ValidationError`	A required env var is missing; the message names it.
OAuth returns to localhost in production	Redirect URL not allow-listed; Supabase fell back to Site URL.
"Add a mailbox" does nothing	Manual linking still off in Supabase.
Gmail stops syncing after a week	Expected: Testing status issues 7-day refresh tokens.
Changing `NEXT_PUBLIC_*` has no effect	Inlined at build time. Rebuild, don't redeploy.
First request after idle is slow	Render cold start — intended, nothing pings it.

Operational notes

- **Groq free tier:** 200,000 tokens/day is the limit that binds, well before requests/day. Bodies are truncated and bulk mail is filtered before the model.
- **"Extract now" is uncapped** — it attempts the entire pending backlog in one request, where sync-triggered extraction caps at 50. On a large backlog it will exhaust the daily budget and likely time out. Prefer "Sync now".
- **Supabase free projects pause** after a week of inactivity; opening the dashboard wakes them.

- **Rotate what passed through chat:** the database password, Groq key, Azure secret and Google secret were all pasted in plain text during setup.

Figures verified at the time of writing: 15 migrations replaying clean from an empty database, 9 tables all with RLS enabled, `anon` and `authenticated` holding zero grants on `public`, Supabase advisors reporting no issues, 400 backend and 197 frontend tests passing.